

Practical 3: Greedy Search Algorithms

```
import heapq
```

I. SELECTION SORT

```
def selection_sort():
    print("\n--- Selection Sort ---")
    n = int(input("How many elements? "))
    arr = list(map(int, input(f"Enter {n} elements separated by space: ").split()))
    print(f"\nOriginal Array: {arr}")

    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
        print(f"Step {i+1}: {arr} → placed {arr[i]} at index {i}")

    print(f"\nFinal Sorted Array: {arr}")
```

II. JOB SCHEDULING

```
def job_scheduling():
```

```

print("\n--- Job Scheduling Problem ---")
n = int(input("Enter number of jobs: "))
jobs = []
print("Enter each job as: JobID Deadline Profit")
for i in range(n):
    line = input(f" Job {i+1}: ").split()
    job_id = line[0]
    deadline = int(line[1])
    profit = int(line[2])
    jobs.append((job_id, deadline, profit))

# Greedy: sort by profit descending
jobs.sort(key=lambda x: x[2], reverse=True)

max_deadline = max(j[1] for j in jobs)
slot = [-1] * (max_deadline + 1)
total_profit = 0
scheduled = []

print("\nScheduling Process:")
for job in jobs:
    job_id, deadline, profit = job
    for t in range(deadline, 0, -1):
        if slot[t] == -1:
            slot[t] = job_id
            total_profit += profit
            scheduled.append((job_id, t, profit))
            print(f" Job {job_id} → Slot {t} | Profit: {profit}")
            break

```

```
print(f"\nScheduled Jobs: {[s[0], s[1]] for s in scheduled}")
print(f"Total Profit: {total_profit}")
```

```
# _____
# GRAPH INPUT HELPER (used by Dijkstra, Prim's, Kruskal's)
# _____
```

```
def input_graph():
    v = int(input("Enter number of vertices: "))
    e = int(input("Enter number of edges: "))
    print("Enter each edge as: u v weight (nodes numbered from 0)")
```

```
graph = {i: [] for i in range(v)}
edges = []
```

```
for i in range(e):
    line = input(f" Edge {i+1}: ").split()
    u, v_node, w = int(line[0]), int(line[1]), int(line[2])
    graph[u].append((v_node, w))
    graph[v_node].append((u, w)) # undirected
    edges.append((w, u, v_node))
```

```
return graph, edges, v
```

```
# _____
# III & VII. DIJKSTRA'S — Single Source Shortest Path
# _____
```

```
def dijkstra():
    print("\n--- Dijkstra's Shortest Path ---")
```

```

graph, _, v = input_graph()
start = int(input("Enter source node: "))

dist = {node: float('inf') for node in range(v)}
dist[start] = 0
prev = {node: None for node in range(v)}
pq = [(0, start)]

print("\nProcessing:")
while pq:
    curr_dist, u = heapq.heappop(pq)
    if curr_dist > dist[u]:
        continue
    print(f" Visiting node {u} | dist = {curr_dist}")
    for nbr, weight in graph[u]:
        new_dist = dist[u] + weight
        if new_dist < dist[nbr]:
            dist[nbr] = new_dist
            prev[nbr] = u
            heapq.heappush(pq, (new_dist, nbr))
        print(f" Updated dist[{nbr}] = {new_dist} (via node {u})")

print("\nShortest distances from node", start)
for node in range(v):
    print(f" Node {node}: {dist[node]}")

# Show path to a specific destination
end = int(input("\nEnter destination node to show path: "))
path = []
curr = end

```

```

while curr is not None:
    path.append(curr)
    curr = prev[curr]
path.reverse()
if path[0] == start:
    print(f'Shortest Path {start} → {end}: {path} | Cost: {dist[end]}')
else:
    print(f'No path found from {start} to {end}')

```

V. PRIM'S MST

```

def prims():
    print("\n--- Prim's Minimum Spanning Tree ---")
    graph, _, v = input_graph()
    start = int(input("Enter starting node: "))

```

```

    visited = set([start])

```

```

    heap = []

```

```

    mst = []

```

```

    total_cost = 0

```

```

    for nbr, w in graph[start]:

```

```

        heapq.heappush(heap, (w, start, nbr))

```

```

    print("\nBuilding MST:")

```

```

    while heap:

```

```

        weight, u, v_node = heapq.heappop(heap)

```

```

        if v_node in visited:

```

```

        continue
    visited.add(v_node)
    mst.append((u, v_node, weight))
    total_cost += weight
    print(f" Added edge ({u} -- {v_node}) weight = {weight}")
    for nbr, w in graph[v_node]:
        if nbr not in visited:
            heapq.heappush(heap, (w, v_node, nbr))

print(f"\nMST Edges:    {mst}")
print(f"Total MST Cost: {total_cost}")

```

```

# -----
# VI. KRUSKAL'S MST — Union Find
# -----

```

```

class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False        # same component → cycle

```

```

if self.rank[px] < self.rank[py]:
    px, py = py, px
self.parent[py] = px
if self.rank[px] == self.rank[py]:
    self.rank[px] += 1
return True

```

```

def kruskals():
    print("\n--- Kruskal's Minimum Spanning Tree ---")
    _, edges, num_nodes = input_graph()

    sorted_edges = sorted(edges) # greedy: smallest weight first
    uf = UnionFind(num_nodes)
    mst = []
    total_cost = 0

    print("\nProcessing edges (sorted by weight):")
    for weight, u, v_node in sorted_edges:
        print(f" Edge ({u} -- {v_node}) weight = {weight}", end="")
        if uf.union(u, v_node):
            mst.append((u, v_node, weight))
            total_cost += weight
            print(" → Added to MST")
        else:
            print(" → Skipped (forms cycle)")

    print(f"\nMST Edges:    {mst}")
    print(f"Total MST Cost: {total_cost}")

```

```
# -----  
# MAIN MENU  
# -----  
  
def main():  
    while True:  
        print("\n" + "="*45)  
        print("  GREEDY ALGORITHM MENU")  
        print("="*45)  
        print(" 1. Selection Sort")  
        print(" 2. Job Scheduling Problem")  
        print(" 3. Dijkstra's Shortest Path (Single Source)")  
        print(" 4. Prim's Minimum Spanning Tree")  
        print(" 5. Kruskal's Minimum Spanning Tree")  
        print(" 6. Exit")  
        print("="*45)  
  
        choice = input("Enter your choice (1-6): ").strip()  
  
        if choice == '1': selection_sort()  
        elif choice == '2': job_scheduling()  
        elif choice == '3': dijkstra()  
        elif choice == '4': prims()  
        elif choice == '5': kruskals()  
        elif choice == '6':  
            print("Exiting. Goodbye!")  
            break  
        else:  
            print("Invalid choice. Please enter 1-6.")
```



```
if __name__ == "__main__":  
    main()
```